第 5 章練習　程序同步

> 本檔只含題目。請先自行建立狀態或變數表，再作答；解答在獨立檔案。

OS05-E01　critical section 性質

某鎖演算法能保證互斥，但候選執行流可被已執行完 critical section 的另一群人無限插隊。

1. 它破壞三性質中的哪一項？
2. 這是否必然造成 deadlock？
3. 如何用有限等待界線修正描述？

OS05-E02　有界緩衝區

buffer 容量為 3，producer 與 consumer 使用 mutex、empty、full。目前 empty=0，full=3，mutex 可用。

1. producer 下一步應先 wait 哪個 semaphore？
2. consumer 完成一項取出後，signal 的順序可以是什麼？
3. 為何不能讓 producer 持有 mutex 後等待 empty？

OS05-E03　條件變數

等待執行緒寫 `if (!ready) wait(cv); use(buffer);`，通知者設定 ready=true 後 signal(cv)。

1. 為什麼 `if` 可能不足？
2. 應改成什麼等待模式？
3. 通知可能在等待前發生，程式如何避免 lost wake-up？

OS05-E04　Peterson 正確性與記憶體順序

兩個 threads 使用 atomic flag[2] 與 atomic turn，初值 flag={false,false}、turn∈{0,1}。鎖演算法依序執行 flag[i]=true、turn=j、while(flag[j] && turn==j) 等待，離場 flag[i]=false。

1. 以 turn 的最後一次寫入說明 mutual exclusion。
2. 證明等待中的 Pi 最多被 Pj 插隊一次。
3. 若所有 shared atomics 使用 sequentially consistent 順序，三個性質為何成立？
4. 若把變數改成非 atomic、允許 compiler 重排，原證明缺少哪一步？

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第 5 章練習　程序同步 （續）

OS05-E05　Semaphore 狀態與 bounded buffer

容量 3，empty=3、full=0、mutex=1。事件依序為 producer 放入 A、B、C，consumer 取出 A，producer 放入 D。

1. 列出每次操作後的 empty、full、mutex 與 buffer 內容。
2. 說明 binary semaphore 保護 critical section 時為何不放行第二個 wait。
3. 若 producer 先等待 empty 失敗卻仍持有 mutex，consumer 是否可能完成取出？用狀態與等待邊解釋。

OS05-E06　Condition variable、Mesa 與 lost wakeup

監視器保護 ready=false 與緩衝區。等待者使用 while(!ready) wait(cv)，通知者取得同一 monitor 後令 ready=true，再 signal(cv)。

1. 說明 wait 為何必須原子地釋放 monitor 並加入 queue。
2. signal 發生在 waiter 檢查 predicate 前，為何不會 lost wakeup？
3. 在 Mesa 語意下，若通知後另一 thread 先把 ready 改回 false，最先喚醒者為何仍不能直接使用 buffer？
4. Hoare 與 Mesa 對 signal 的成本與正確性條件差異為何？

OS05-E07　被通知者沒有拿到唯一項目

採 Mesa 語意、一把 m、condition variable cv；q 初始空。C1、C2 各執行 `lock(m); while(q.empty()) wait(cv,m); pop(q); unlock(m)`；P 僅執行一次 `lock(m); push(q,A); signal(cv); unlock(m)`。本例無 spurious wakeup，mutex 不保證 FIFO。

事件依序為 T0：C1 wait；T1：P 持 m 放 A；T2：P 仍持 m 並通知 C1；T3：P 釋鎖後，C2 完整執行 get 並釋鎖；T4：C1 重取 m；T5：C1 的下一步。

1. 按 T0 至 T5 的六個事件，列 m owner、q、CV 等待者、已通知但未重取鎖者 E；最後一個事件是 C1 的下一步。
2. 若把 while 改成 if，第一個錯誤操作在哪裡？本例為何不能歸因於 spurious wakeup 或 lost wakeup？
3. 若自行拆成先 unlock、稍後才登記等待，構造 q 非空但 C1 睡著的三步交錯。它是否為合法的 pthread_cond_wait 呼叫方式？
4. 若 C1、C2 都已 wait，P 放一個 A 後改用 broadcast，兩人能否各取一項？while 是否保證公平或未來一定完成？

第 5 章練習　程序同步 （續）

參考資料與署名

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study